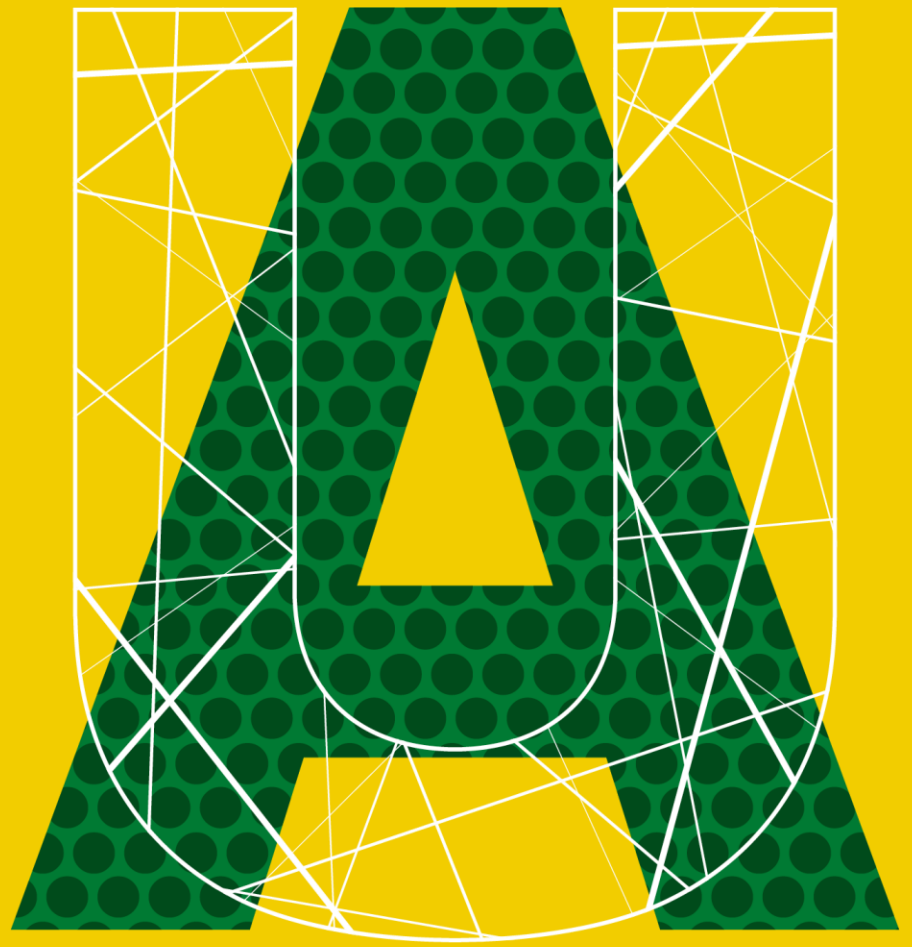
Efficient Robotic Pushing with Diffusion Policy

Adaptive Execution and Self-Imitation-Based Fine-Tuning

Qihang Feng

UNIVERSITY OF ALBERTA

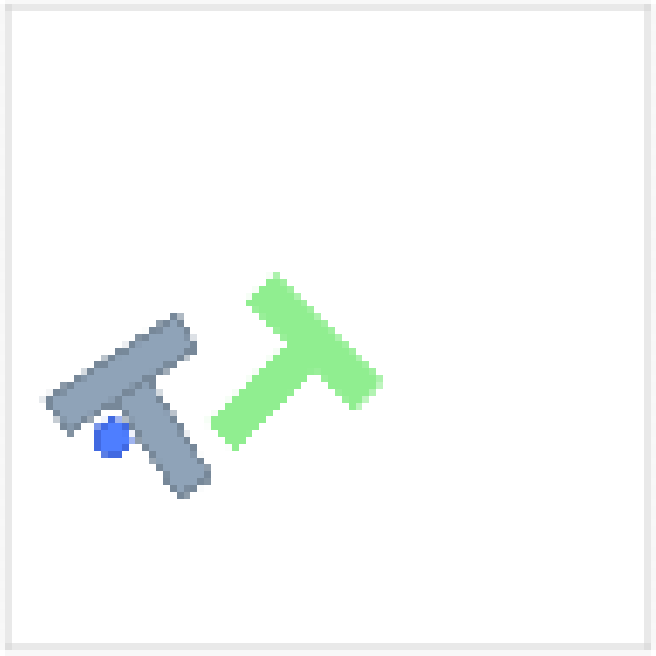


ECE 730 PROJECT MID-UPDATE

Computation and AI for Cyber-Physical Systems

PushT task and feedback loop

Push the T-block onto the green target



Recorded camera image

1. Observe

Two camera frames and pusher position

2. Predict

16 action positions from Diffusion Policy

3. Execute

8 or 12 movements with visual monitoring

4. Observe again

The movements change the camera image

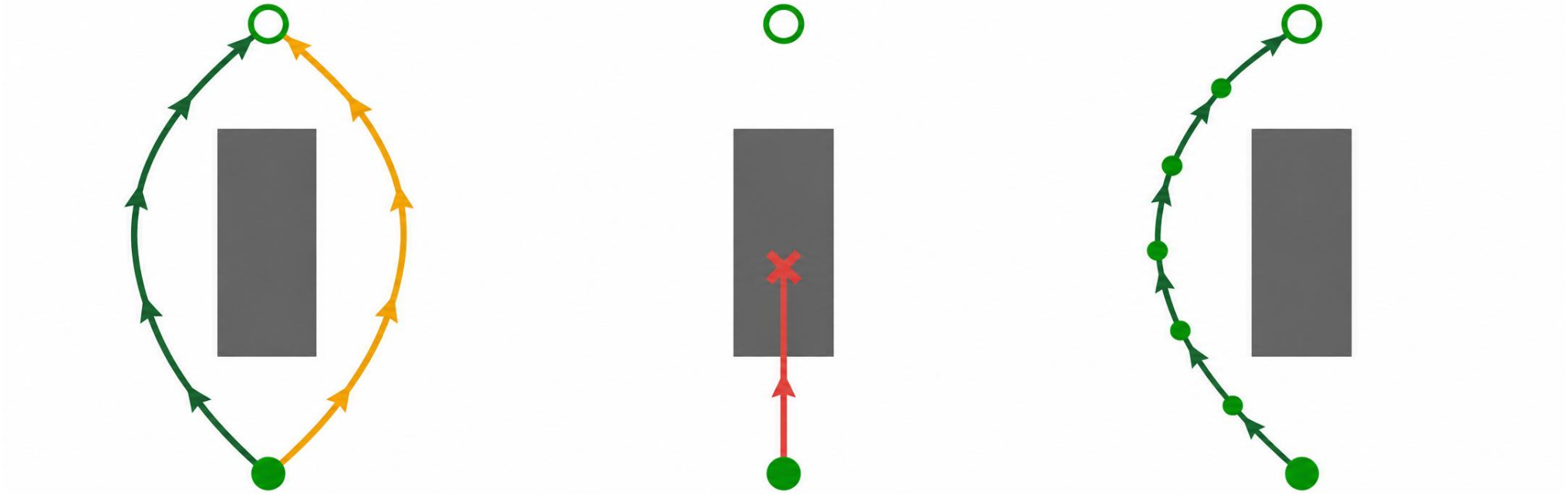
Actions specify absolute x–y pusher targets

Success requires >95% target coverage within 300 movements. Simulator geometry supplies scoring.

Why Diffusion Policy?

The same scene can support several valid movement plans

Demonstrated routes **Averaged route** **One diffusion sample**



Diffusion can represent different choices and sample one coherent sequence

A public PushT checkpoint supplies sequences for reuse and limited fine-tuning

Conceptual obstacle-avoidance example. Chi et al., Diffusion Policy (RSS 2023), Sections 4.1–4.3.

Related work and limitations

Execution and recovery methods that motivate our PushT study

Work	Main contribution	Reported limitation
Diffusion Policy (2023)	Predicts action sequences from expert demonstrations	Needs adequate demonstrations. Iterative denoising adds latency.
SGAC (2025)	Self-guidance and adaptive execution	Compares fresh plans at each step. Cost is similar to closed-loop control.
ReGuide (2026 preprint)	Fine-tunes on successful recovery rollouts	Needs tuning for each task. Broader evaluation across training runs and real robots remains open.

We study whether camera-guided reuse and a small policy update can improve pushing success while reducing prediction calls

RL performance and planning costs

Reported PushT-Vision success after 3 million training interactions

Method	SAC	TD3	QSM	DPMD	DreamerV3	TD-MPC2	GTP
Success (%)	0 ± 0	0 ± 0	0 ± 0	0 ± 0	0 ± 0	**97 ± 1**	**99 ± 1**

Balim et al., GTP (2026 preprint), Table 5. Mean ± standard error over 5 training seeds.
Velocity commands and IoU scoring differ from our evaluation.

Benchmark result

The first five methods recorded 0% success under this task and training protocol.

Planning cost

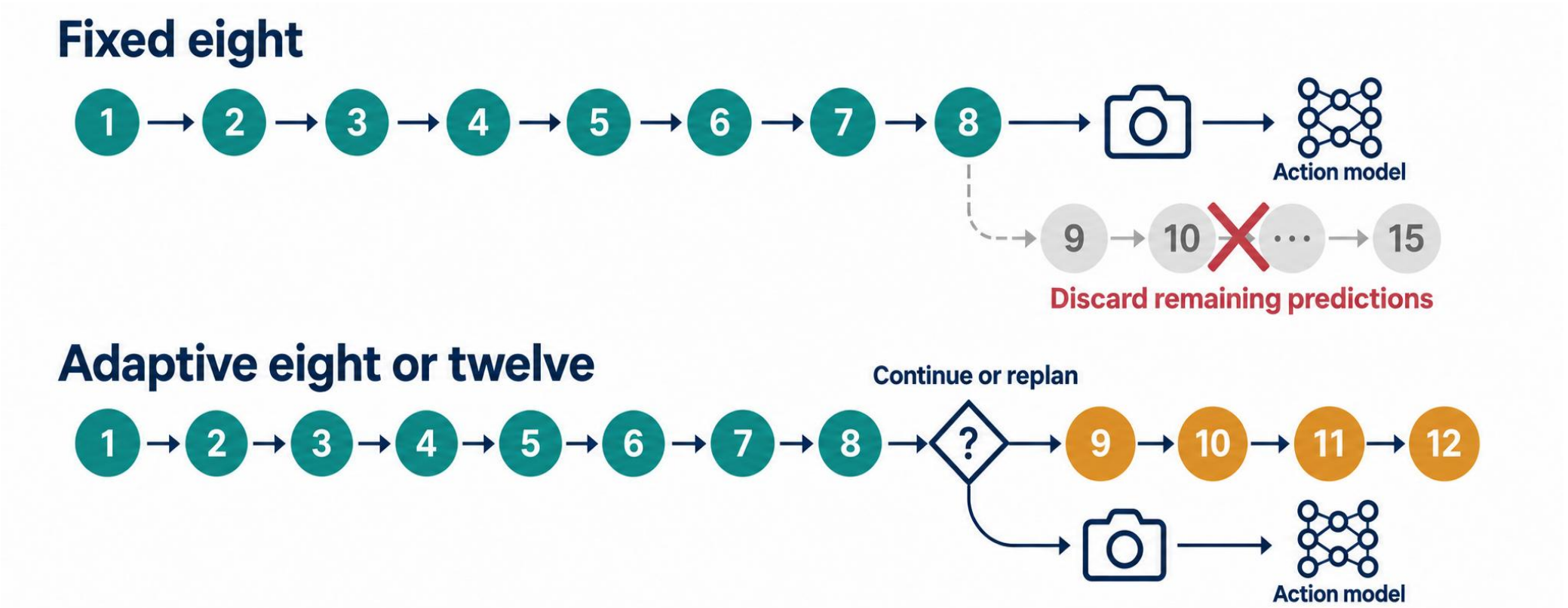
TD-MPC2 and default GTP replan each movement, using learned models to evaluate candidate plans.

Table 6 reports higher action-selection time than SAC on the authors' RTX 5090.
GTP also tests planning-free and multi-step execution.

A runtime advantage for our method requires a matched measurement.

Adaptive execution

After movement 8, camera and path checks decide whether to continue to 12.

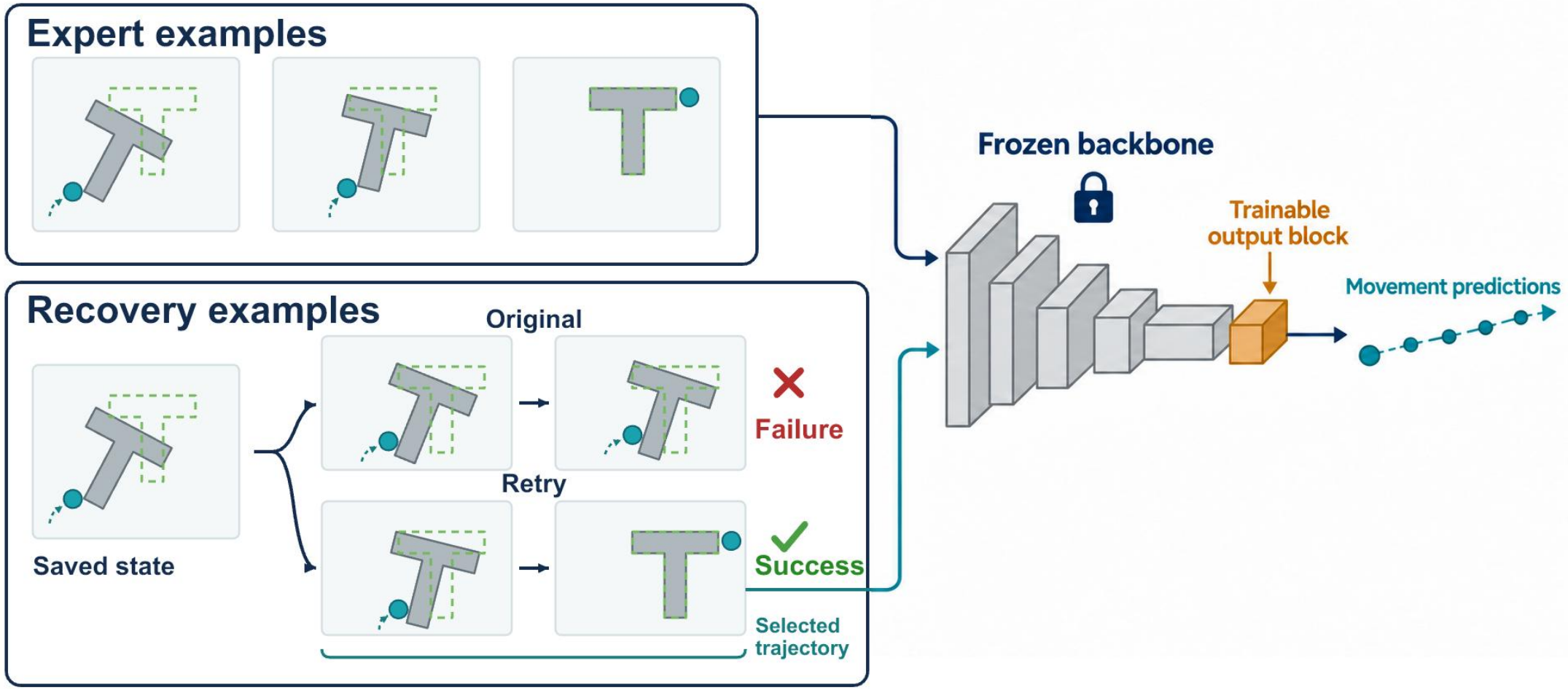


Each call generates 16 actions. The controller changes how many we execute.

Camera alignment history and the remaining path guide continuation and bounded interruptions.

Recovery fine-tuning

Mix expert demonstrations with successful retries of failed pushes.



600 updates to the final convolution block with expert rehearsal

Four matched configurations

40 starting scenes, 2 prediction draws each, 80 attempts per configuration

Configuration	Success	Alignment	Predictions	Time (s)
Original / fixed 8	57.5%	90.96%	29.03	143.26
Original / adaptive	60.0%	91.59%	25.34	135.95
Fine-tuned / fixed 8	70.0%	92.01%	27.66	136.07
Fine-tuned / adaptive	**75.0%**	**91.88%**	**22.51**	**120.92**

Adding adaptive execution to the fine-tuned model saves 18.6% of predictions

Its additional success gain is +5 percentage points, with a 95% interval of −8.75 to +18.75.

Alignment, predictions, and time are attempt averages including failures. Timing spans collection sessions.

Standard RL comparison with visual SAC

All policies use the same 40 starting scenes, observations, and commands

Policy	Successful pushes	Final alignment
Visual SAC, 100k interactions	0/40 (0%)	6.37%
Original / fixed 8	46/80 (57.5%)	90.96%
Fine-tuned / adaptive	**60/80 (75.0%)**	**91.88%**

Each Diffusion Policy configuration runs twice per scene. SAC runs once.

One SAC training seed, 100,000 online interactions, and about 67 minutes on this laptop

These results describe the saved checkpoint at this training budget.

Gained and lost baseline successes

Compared with original / fixed eight: 46/80 successful pushes

Policy	Gained	Lost	Net gain	Successful pushes
Original / adaptive	19	17	+2	48/80
Fine-tuned / fixed 8	18	8	+10	56/80
Fine-tuned / adaptive	**23**	**9**	**+14**	**60/80**

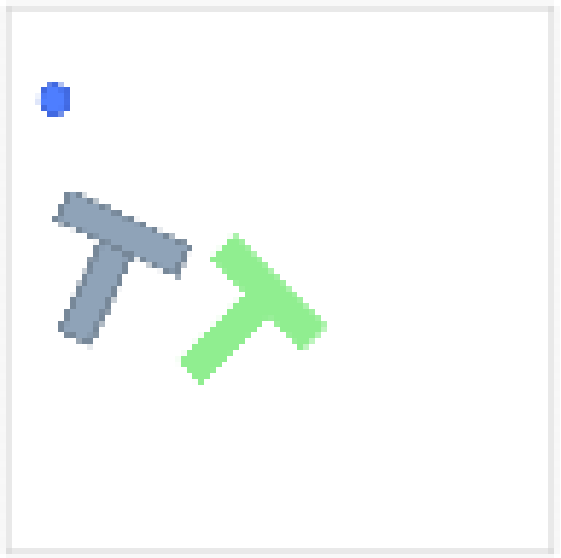
Gained: baseline fails, method succeeds. Lost: baseline succeeds, method fails.
80 paired attempts = 40 starting scenes × 2 prediction seeds.

Next step: investigate the 9 lost baseline successes and test recovery ideas on separate evaluation scenes

A saved improvement

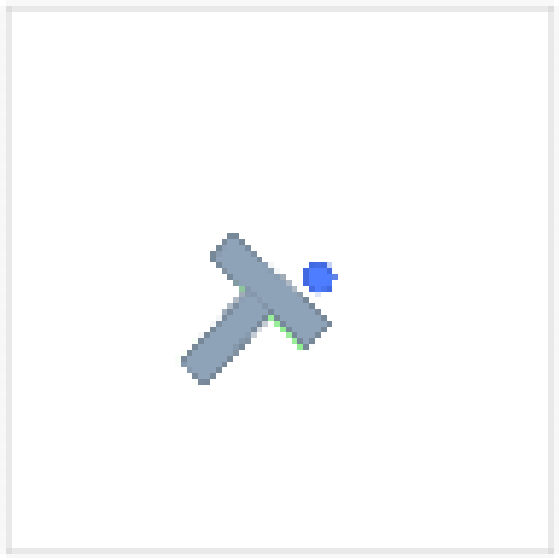
Saved scene 201, prediction draw 60000

Initial scene



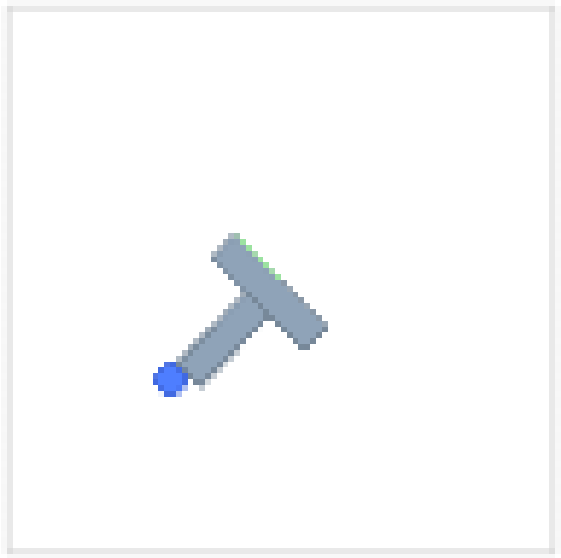
Same starting scene
and initial prediction seed

Original / fixed 8



92.21% coverage
300 movements
38 predictions

Combined method



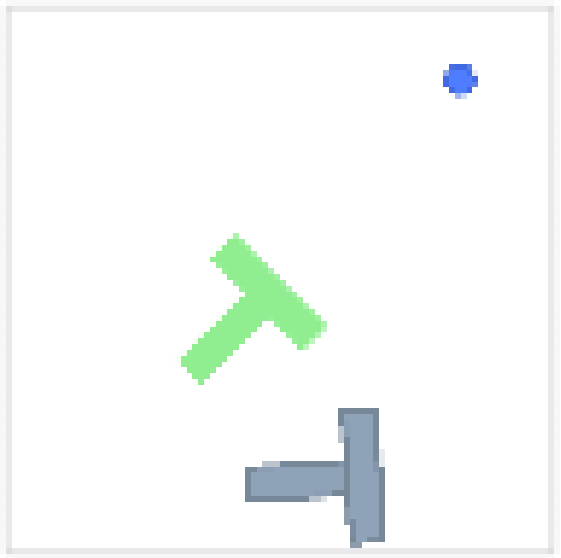
97.14% coverage
173 movements
20 predictions

Combined method means fine-tuning with adaptive execution. This is one selected example.

A lost baseline success

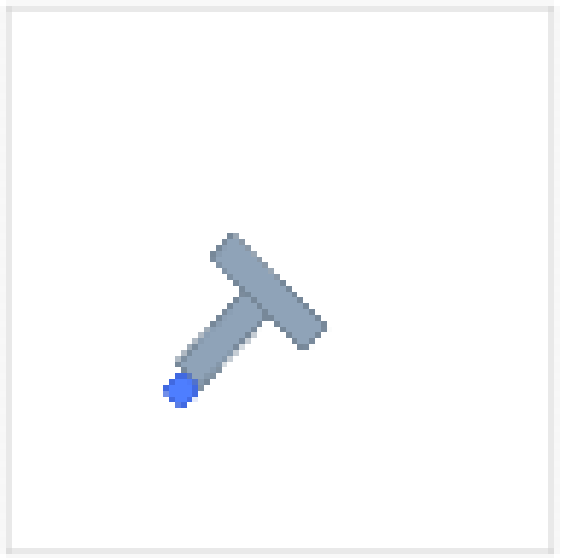
Saved scene 179, prediction draw 60000

Initial scene



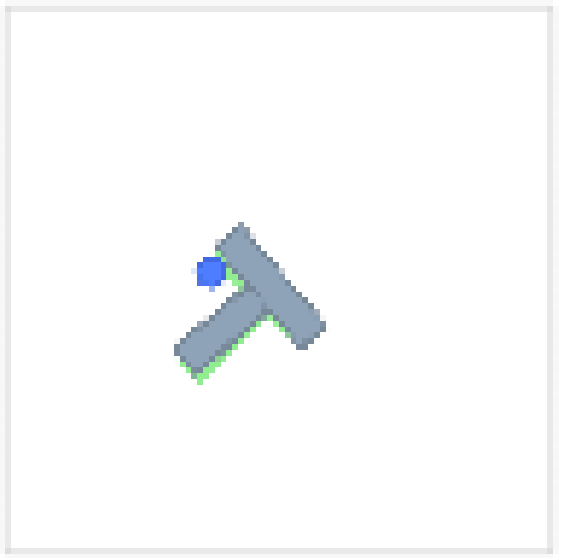
Same starting scene
and initial prediction seed

Original / fixed 8



97.53% coverage
177 movements
23 predictions

Combined method



81.78% coverage
300 movements
34 predictions

Evidence covers one task and one fine-tuning seed. Recorded runtime spans collection sessions.

Remaining work and professor feedback

Remaining work

Investigate the 9 attempts where our full method loses a baseline success

Test any recovery change on separate scenes before claiming improvement

Finish the 7-page report, reproduction instructions and final presentation

Feedback for this meeting

Is the present experimental evidence sufficient for the course scope?

Should recovering lost baseline successes be the next research priority?

Final presentation Dec. 2. Final report Dec. 13, according to the project guideline.

Thank you!



UNIVERSITY
OF ALBERTA